VU AI HUB · LECTURE NOTES

Intelligent Systems

Year 1, November to December 2025

Written by Inaaya and shared on Notion. Original page: cheddar-wall-f3b.notion.site/2999801a93098092bb56c73a4c44a58f

Contents

1. 6th of October (Methodology) Intro to AI
2. 27th of October (HC)
3. 28th of October (HC)
4. 29th of October (WC)
5. 3rd of October (HC)
6. 4th of November (HC)
7. 5th of November (WC)
8. 10th of November
9. 11th of November (HC)
10. 12th of November (WC)
11. 17th of November (HC)
12. 18th of November (HC)
13. 19th of November (WC)
14. 22nd of November
15. 18th of November (HC)
16. 25th of November (HC)
17. 26th of November (WC)
18. 1st of December (HC)
19. 2nd of December (HC)
20. 3rd of December (WC)
21. 8th of December (HC)
22. 9th of December (HC)

6th of October (Methodology) Intro to AI

Discrete Mathematics

Naive Set Theory

A set is a structure representing an *unordered* collection of zero or more *different* objects

- set theory deals with operations between, relations among, and statements about sets
- Notation: { }

We use variables, ex. S, T, U etc.

- We denote a set S in curly brackets with all the variables inside: S = {a, b, c}

Set builder notation: for any proposition P(x) (statement), {x/P(x)} is the set of all x such that P(x)

- ex. {x | x is an integer where x > 0 and x <5} = {1, 2, 3, 4}

Sets are unordered so {a, b, c} = {b, c, a} = {a, c, b}

Sets are distinct (unique) so {a, b, c} = {a, a, b, b, b, c}

Sets can be never ending (infinite)

Venn Diagrams = model of sets

The universe (of discourse) *U* is the set of all things

x ∈ S means “x is an element or member of set S”

- “x is in S”

Set A is equal to set B *iff* they have the same exact members

- x ∉ S *iff* x is not in S

∅, null or empty set, is the unique set that contains no elements

The symbol for subset is ⊂ (proper subset) or ⊆ (subset or equal to)

- S ⊆ T, every element of S is also an element of T
- S ⊂ T, every element of S is in T, but not the other way around

The symbol for superset is ⊃ (proper superset) or ⊇ (superset or equal to)

- S ⊇ T, every element in T is in S, but not necessarily vice versa

The elements in a set may be sets themselves

|S| means "the cardinality of S, or the measure of how many elements S has"

- |{1, 2, 3}| = 3
- |∅| = 0
- |{1}, {1{2, 3}}| = 2

The power set P(S) of a set S is the set of all subsets of S

- P(S) = {x | x ⊆ S}
- Sometimes P(S) is written as 2 to the power of S

A ∪ B means "A union B", or the set containing all elements that are either in A or in B

- A = {1, 2, 3} and B = {4, 5, 6} means A ∪ B = {1, 2, 3, 4, 5, 6}

A ∩ B means "A intersection B", or the set containing all elements that are both in A and B

- A = {1, 2, 3} and B = {3, 4, 5} means A ∩ B = {3}

A ∩ B = ∅ means "A and B are disjoint", or their intersection is empty

- A = {1, 2, 3} and B = {4, 5, 6} means A ∩ B = ∅

A − B means "the difference of A and B", or the set of all elements in A but not in B

- A − B = {x | x ∈ A and x ∉ B}
- Also called "the complement of B with respect to A"

A' is "the complement of A", or everything in the universe that is not A **...?**

Identity
Domination
Idempotent
Double complement
Commutative
Associative
De Morgan

Relations

A list is a structure representing an *ordered* collection of zero or more objects which may include *duplicates*

- Notation: [ ], ( ), ⟨ ⟩
- Elements in a list have positions
- Finite lists have a length

Cartesian product of two sets, a set of pairs (lists of length x 2)

- A x B = { ⟨a, b⟩ : a ∈ A and b ∈ B}
- A x A = A to the power of 2

N-tuples are sequences of lengths n, whose order matters

- (a) is a singles
- (a, b) is a pair
- (a, b, c) is a triple

Relations are sets of n-tuples

A relation in the set A is a binary relation of type A to the power of 2

A binary relation is a relation of type A x B or A to the power of 2

- FollowsCourse = {⟨x, y⟩ : x follows y} ⊆ Students x Courses
  - Alternative infix notation: x FollowsCourse y instead of ⟨x, y⟩ ∈ FollowsCourse

---

Graphs

A graph G is generalisation of the simple concept of a set of dots, links, edges, or arcs

- G = (V, E), where V is the set or vertices (singular: vertex), and E is the set of paired vertices or edges

Undirected graphs

- A non-empty set of vertices V, and a set of unordered, distinct edges E
  - G(V, E) V = {u, v, w} and E = {{u, v}, {v, w}, {u, w}}

Directed graphs

- A non-empty set of vertices V, and a set of directed edges E, consisting of ordered pairs
  - G(V, E) V = {u, v, w} and E = {(u, v), (v, w), (u, w)}

Definition: A graph is Eulerian if it is both connected and has a closed trail (a walk with no repeated edges) containing all edges of the graph

Theorem: A graph is Eulerian if every node in the graph has an even degree

The adjacency matrix of a graph G is the matrix A defined as adjacency between nodes u and v

- if the set of edges E includes (u, v), then A[u, v] has a value of 1
- if there is no direct path to u to v, then A[u, v] has a value of 0

Trees

A tree is a connected, undirected graph with no simple circuits

- A circuit (cycle) is a path of length ≥ 1 that begins and ends at the same vertex
- Ex. tournaments and family trees

Once a tree has a designed root node, it is possible to assign a direction to each edge

- Root node is the one at the top
- Internal nodes are the nodes in between
- Parent relations, one above the other
- Sibling relations, the same order or the same parent
- Leaf node is the last node

A rooted tree is an n-ary tree if every internal vertex has no more than n children

- A tree is full n-ary if every internal vertex has exactly n children
- An n-ary tree with n=2 is called a binary tree

The level or depth of a vertex v in a rooted tree is the length of the unique path from the root to this vertex

The height (also called depths) of a rooted tree is the maximum of the levels of vertices

27th of October (HC)

Lecture 1: Introduction

Core Topics

- Decision making
- Integrating needed knowledge to make the right decision
- Machines need to adapt and learn

Schnapsen

- Watch a video

The agent function maps percent sequence to actions

- f: P* → A

The agent function will internally be represented by the agent problem

The agent program runs on the physical architecture to produce f

A rational agent (could be humans, but not in this case) chooses whichever actioin maximizes the expected value of the performance measure given the percept sequence to date and prior environment knowledge

Rationality depends on:

- Expected value of performance measure (Heuristics)
- Actions and choices (Search)
- Prior environment knowledge (KR)
- Percept sequence to date (Learning)

Rationality is not the same as omniscience or perfection

To design a rational agent we must specify its task environment:

- Performance (what it does)
- Environment (where it does it)
- Actuators (how it does it)
- Sensors (what it perceives)

→ PEAS (acronym)

A state space is an abstraction of the real world which serves to simplify a problem

Real state → abstract state (graph) + Real action - abstract action = Real path that solves a real-world problem → abstract solution

*One real world problem may have many alternative state space representations

All problems can be reduced into nodes (states) and vertices (actions)

28th of October (HC)

Lecture 2: Uninformed search

State space representations

- All the important graph representations you need to solve a problem

---

Search Algorithms

Searching the state space by generating a tree

- Initial state is the tree root
- Nodes and leaves generated through successor functions
- Depth of a node is its distance from the root

Initialize the search tree to the initial state of the problem

if no candidates to expansion then return failure

choose leaf node for expansion according to strategy

if node contains goal state then return solution

else expand the node and add resulting nodes to the search tree #determines search process

- A state is a representation of a physical configuration
- A node is a data structure specifically belonging to a search tree
  - multiple nodes in the same tree can correspond to the same tree
- A tree has parents and children, and includes path cost and depth

---

Uninformed Search

Uninformed (blind) search uses only information which is available in the problem definition

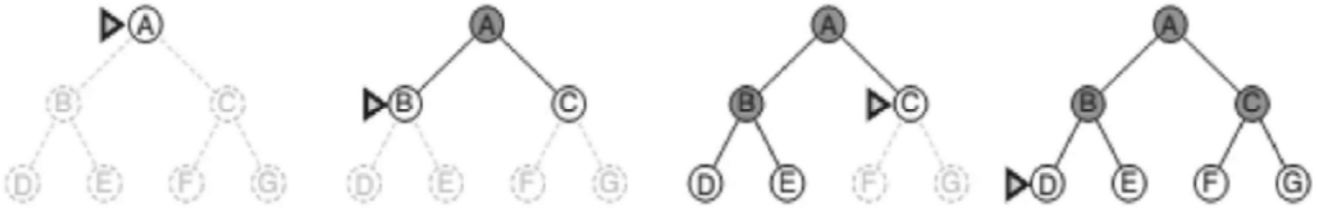
→ no strategy to determine one non-goal state is better than another

A *fringe* is a data structure similar to a list, which stores all of the relevant nodes while the algorithm still builds the tree, the nodes that still need to be expanded

Breadth-first search

Method: expand the shallowest unexpanded node

Implementation: *fringe* is a first-in-first-out (FIFO) queue



C needs to come before D and E. If D is the solution, then you didn't find it while exploring B (which gives you D and E), it'll go in the FIFO stack so it'll come after exploring C (which gives you F and G).

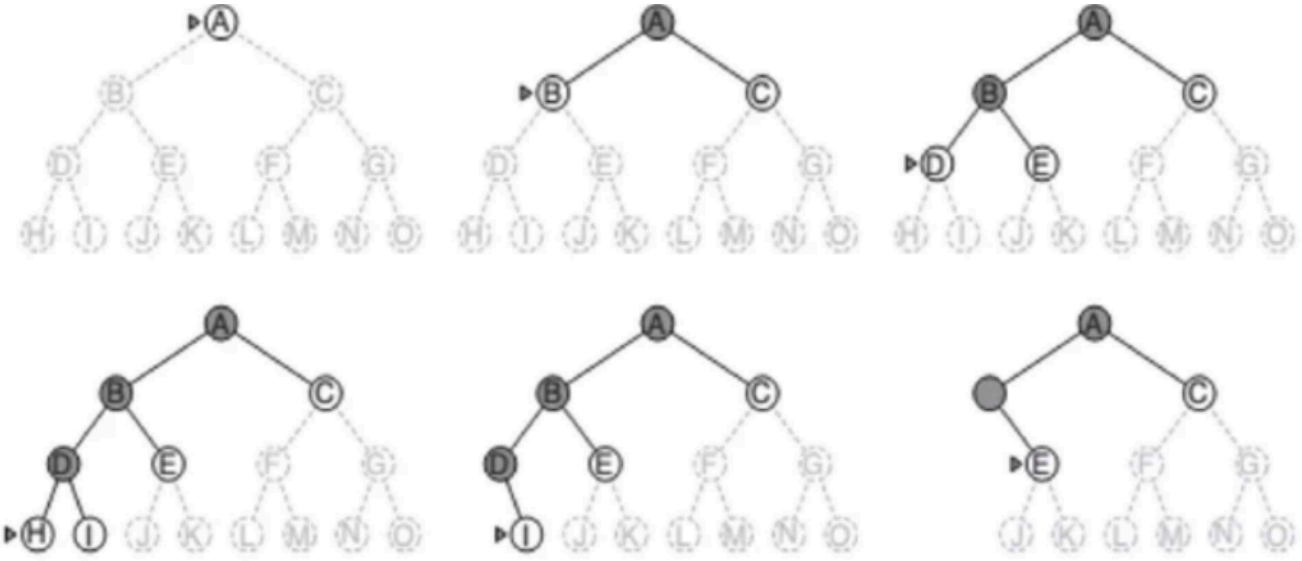
You will always find a solution and the most optimal one, it does depend on if the branching factor is finite

Depth-first search

Method: expand the deepest unexpanded node

Implementation: *fringe* is a last-in-first-out (LIFO) stack

- The computer could get stuck in this scenario when there are too many letters while trying to get a level back up
- If there are no children left you back-track, go back one step, not to the node you came from but to another node on a higher level.



You go all the way down and if there is no solution found you backtrack. If H is not the solution you backtrack to I and then to E. After J and K you go back to C.

A strategy defines node expansion order

- Completeness → does the strategy always finds a solution if one exists?
- Optimality → does it find the least-cost solution?
- Time complexity → what number of nodes do we look at, are generated/expanded?
- Space complexity → what number of nodes do we need stored in memory during search? The longer the chain of descendants, the higher the space complexity

Time and space complexity are measured in terms of

b the maximum branching factor of the search tree

d the depth of the least-cost solution

m the maximum depth of the state space problem (can be infinite)

- they keep checking the same things all over again because repeating is possible

Depth-limited search

*The same as depth-first search but with limited depth *l*

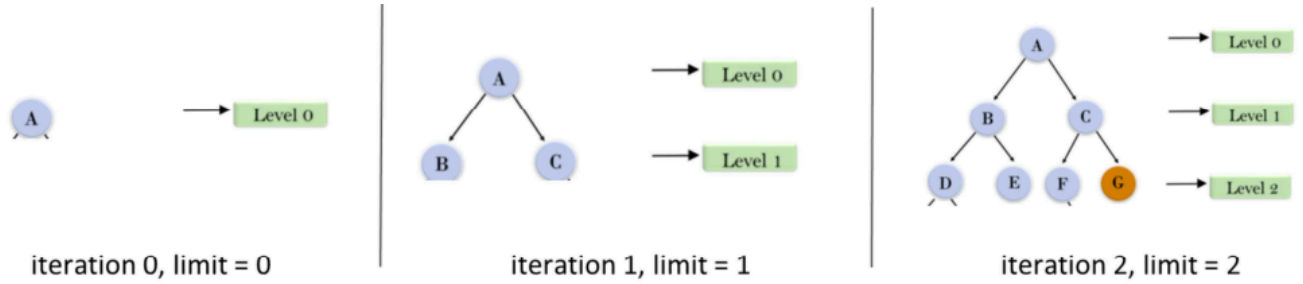
Method: expand the deepest unexpanded node, but nodes at depth *l* have no successors

Implementation: if *l* < d → incomplete search if *l* > d → not optimal

Iterative Deepening Depth-First Search

*A strategy to find the best depth limit *l* without getting stuck in infinite loops like in DFS

- Each consecutive search run adds one depth level
- Often used in combination with DFS
- Combines benefits of DFS and BFS to combat the memory problem



You have to write everything you already explored as well: A - ABC - ABCDEFG. You don't have to explore every letter it just has to be in the result list.

Performance Measure	Breadth-first	Depth-first	Depth-limited	Iterative Deepening
Completeness (finding the goal state)	Yes*	No	Yes, if *l* > d	Yes
Time	b^d	b^m	b^l	b^d
Space	b^d	bm	b*l*	bd
Optimality	Yes*	No	No	Yes

*It only finds the best solution when the branching factor (the amount of children) is finite

---

Search Directions

- Data driven starts at the current state
  - when there are too many goal states, and when they're not explicitly known
- Goal driven starts at the goal state
  - when the branching factor is larger when starting from the initial state

Search direction is a relative concept that depends on how you represent the state-space

29th of October (WC)

Initial state = state that the agent starts in (ex. no region is colored)

Goal test = test which determines whether a given state is a goal state (ex. all colored + no adjacent)

Successor function = agent's options: given a state, it returns a set of <action, state/successor> pairs, where each state is reachable by taking the action (ex. assign a color to a region)

Cost function = a mathematical formula used to predict **???** (ex. number of assignments)

If two parents go to the same child, ex. F and G both go to M, and you have to explore M twice you write the second M in brackets, (M), in the result list.

You don't stop randomly in a level, you write all the letters in the same level down, even if the goal state is the first letter in that level.

3rd of October (HC)

Lecture 3: Informed Search

Informed search uses an intelligent method to decide which nodes to expand next called heuristics

A heuristic *h(n)* determines the evaluation function *f(n)* that an informed search algorithm uses, it is:

> *A rule of thumb, simplification, or educated guess that reduces or limits the search for solutions in domains that are difficult and poorly understood*

h(n) is calculated for every node in the search tree: it's the estimated cost of the cheapest path from the current node *n* to the goal node *g*

→ if *n* = *g*, then *h(n)* = 0 because we are at the goal itself

Best-first search

An informed search method that expands the node with the most promising outcome

Method: expand to a node based on evaluation function *f(n)*, which measures the distance from the node to the goal

Implementation: fringe is a queue sorted in decreasing order of desirability, lower distance to the goal = higher desirability

Manhattan Distance

Where you ignore all the constraints that are in the way, you simply count how long it will take you to go from A to B using top, down, left or right. Typically used when you have a grid.

- $f(n) = g(n) + h(n)$

Straight-line-distance (SLD)

A heuristic which is the cost of a straight line towards the goal, ex. how a bird would fly

---

Greedy best-first search

Expands the node with minimal heuristic function *h(n)*

- Algorithm greedily chooses a new successor, only according to the estimate of distance to the goal
- The evaluation function *f(n)* = heuristic *h(n)*

Greedy Search Evaluation

Greedy Search is incomplete even in a finite search space

- Repeats nodes that have already been expanded, ending up in cycles

Not optimal because it can lead to false starts

Time complexity is b^m, all nodes kept in memory

Space complexity is b^m, good heuristic dramatically improves this

---

Global vs. Beam (Local)

Global greedy search keeps all the open nodes in the fringe, everything we discussed before

Efficiency can be increased by restricting the size of the fringe, Beam Search, which is local

Local Beam Search

Expands to the most promising nodes, only expanding the *k* best nodes at each level

Method: at each state, choose most promising state according to a heuristic

Implementation: keep *k* states, use breadth-first search to traverse each level's *k* best nodes of all successors

Hill Climbing

Compares neighboring nodes and selects the one which optimizes the current cost

Method: at each state, choose the most promising state according to a heuristic

Implementation: limit the fringe to 1 element, never increase *f*

State Space Landscape

A useful graphical representation of an algorithm's current state *x* and the objective (cost) function *y*

Local Beam Search Evaluation

Choosing top *k* states of all successors is not the same as *k* states run in parallel

→ Searches that find good states recruit other searches to join them

---

A Search

The most well-known form of best-first search which evaluates nodes by combining two costs

g(n), the cost to reach the node from initial node

h(n), the cost from node to goal from current node (heuristic)

→ $f(n) = g(n) + h(n)$

where *f(n)*, estimated total cost of path through *n* to the goal

A* Search

This is A Search with an admissible heuristic, meaning it never overestimates the cost to reach the goal

Admissible heuristics are optimistic:

$h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost from *n*

$h(n) \geq 0$ meaning $h(G) = 0$ for any goal *G*

ex. h ^ SLD(n) can never overestimate the actual road distance

A Search Evaluation*

A* search will follow the same path as A search when they use the same heuristic; the difference is whether the heuristic is admissible

A* is optimal and will find the cheapest path because the h(n) is admissible

It is also complete because it will find the goal state if one exists

Time complexity is exponential with path length and depends on the heuristic

Space complexity is high because all nodes are stored

Thus, computationally expensive but effective

4th of November (HC)

Lecture 4: Adversarial Search

Search with Partial Information

If an agent only has partial information the environment is not fully observable or deterministic, this is the case for sensorless problems:

- No knowledge of current state
- Agent may perform unnecessary actions

Solution: let agents have *belief states*, the agent's current belief about possible physical states it may be in, in here all member states are goal states

- s physical states = 2^s belief states

Games

We can define the environment of a game in terms of how much information we have and how predictable it is. Deterministic means that a game is fully dependent on the actions of each player, while stochastic means there is some uncontrollable element, ex. rolling a dice.

If we have perfect information, this means that the agent can see all relevant factors of the current state to the decision or action it has to take.

- ex. Schnapsen Phase 1 has imperfect information because the agent cannot see all the cards in the deck.

Games are a form of competitive, multi-agent environments. Agents try to win over their adversary (opponent), giving rise to adversarial problems:

- What are other players doing? How do their actions have an impact on mine?

Search Problems vs Games

Search Problems	Games
No adversary	Adversary
Solution = action plan to reach goal	Solution = strategy for every new move
Heuristics can find the optimal solution	Time limit forces approximate solution
Actions taken are defined by optimal solution	Evaluation function for quality of current game position

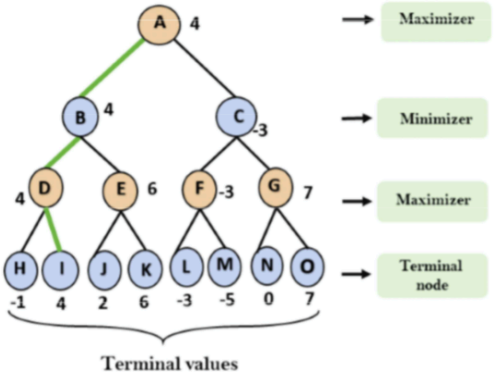
---

Minimax

An adversarial search method used for games that uses two players called min and max

- Each layer of the search tree is either min or max's turn

Method: Start with the leaf node values and backtrack up through the tree to find the optimal move for each player



You go from bottom to top

MAX tries to maximize the value. MIN tries to minimize the value, the assumption is that both players play optimally

- If MIN or MAX doesn't play optimally, then the other player will perform even better

---

Utility vs Heuristics

Utility means that value is based on quality of the state

Heuristic evaluation means value is based on estimation of the quality of the state. In a board game, this function calculates the value of the board depending on piece placement.

Minimax requires many leaf-node evaluations, which is impractical if search is to take a reasonable amount of time

---

Horizon effect

???

5th of November (WC)

Hill Climbing

When 2 nodes have the same, most promising, value, you choose from alphabetical order

You stop when you found the lowest value, you don't have to finish the tree, which means you don't always reach the goal

You only use the estimate in hill climbing, from the goal state to where we are

Greedy best first search

You don't put the letters in the fringe in the order that you encounter them, you do it in the order of their heuristic values, so the smallest values come first.

Manhattan Distance

You just expand the lowest value, so when you already expanded D5 tot C5 and E5 for example, and C4 is on the same level as D5 but is has the same heuristic value as C5 then you choose on alphabetical order and don't explore C4 anymore. But when later on you have nodes that have a higher value than E4, then you go back and explore E4. When a child is already in the tree, you don't write it down twice. The solution is the order you expanded the nodes, not the fastest one in the end.

f(n) keeps the walls in mind but h(n) is an estimate so you don't look at the walls and can go through them

*A**

You don't stop when you found your goal, you stop when you found the best path to the goal. That doesn't mean you have to expand every node, this can happen when a node is never the lowest value.

When they ask for the A* path you put the order you expanded the nodes in the solution, not the fastest route in the end, but when they ask for the best path A* found you put the fastest route in the solution.

Heuristic is admissible when you can not reach the goal in less steps than the highest estimate.

The estimated distance is always an underestimation of the real distance (and makes this algorithm therefore admissible)

Minimax

Recursive function is when the function calls itself inside the function, which creates a loop without for or while

Alpha-Beta Pruning

Condition for pruning Alpha >= beta

Alpha = - infinity

Beta = + infinity

Whenever you do the maximum you look at the alpha, whenever you do the minimum you look at the beta

Random

Heuristic value is the path you take to get from the initial state to the goal

You can also go up, it doesn't make sense because it's not the way to the goal, but you should write them down in the tree

10th of November

Lecture 5: Logical Agents

Logical reasoning over game states

Logic is used to represent an agent's environment in a declarative way

Declarative means using explicit statements to say something about the state of the world

We use logic in AI to:

- Deal with partial or implicit information
- Knowledge about environment and tasks
- Reason over possible states
- Represent human knowledge

Logics and formal systems

A knowledge base is a collection of statements, propositions (which can either be true or false), which we know to be true

What we need for a Logic or Formal system

- Syntax: the rules or language used for constructing and transforming symbols and operators in propositions
- Semantics: interpretation of the meaning symbols and operators in propositions
- Calculus: processes for determining whether statements are true according to our semantics

---

Propositional logic

Syntax

Propositions: statements (symbols + operators)

Symbols: things in the universe, ex. $P_{1,2}$

Operators: and, or, not, implies, equivalent, ex. $P_{1,2}$ or ex. $P_{2,1}$

Semantics

Propositions can be True or False, ex $S_{1,2}$ thus $W_{1,3}$ or $W_{2,2}$

Calculations

Next Lecture

Propositional logic syntax

P_1 and P_2 can be used as sentences, by using the next symbols in between them

- Negation ¬
- Conjunction ∧
- Disjunction ∨
- Implication ⇒

- Biconditional ⇔

Propositional logic semantics

A sentence can either be true or false
Complex statements are compositions of simple statements. Their truth value is calculated from the meaning of its parts

- Conjunction ∧ *and*
  - only true when both sides are true
- Disjunction ∨ *or*
  - true when at least one side is true
- Implication ⇒
  - true when both sides are true or both sides are false
  - also true when the right side is true
- Double implication ↔
  - only true when both sides are true or both sides are false

---

A knowledge base is a set of sentences that we assume to be true, written in curly bracket notation

Models are interpretation functions of the variables, or validations, which evaluate all formulas in a KB to true

- In other words, truth values that make a knowledge base true

---

SAT Solving and Entailment

A sentence is satisfiable if it is true in some interpretation, there exists a model

A sentence is unsatisfiable if it is true in no interpretation, there is no model

A sentence is valid if it is true in all interpretations, also called a tautology

A sentence is entailed by a KB, i honestly don't know what I'm seeing ???

Satisfiability Solving

It is the question: "Is there a model?", the task of finding one or more models of the KB

It is an old but still active field of research, used in hardware/software verification, planning & scheduling, control, protocol design, etc.

Entailment as Satisfiability

We can check what follows from a knowledge base by checking unsatisfiability, or nonexistence of a model

11th of November (HC)

Lecture 6: Propositional Logic

CNF (Conjunctive Normal Form)

An AND or OR propositional logic formula, it is the standard form used in SAT solvers and DPLL algorithms. Transform any propositional formula into CNF:

Eliminate Equivalences

(A ↔ B) is equivalent to (A → B) ∧ (B → A)

(A → B) is equivalent to (¬A ∨ B)

Move NOTs inward (De Morgan's law)

¬ (A ∧ B) is equivalent to ¬A ∨ ¬B → Not and is or

¬ (A ∨ B) is equivalent to ¬A ∧ ¬B → Not or is and

Distributive law (distribute OR over AND)

A ∨ (B ∧ C) into (A ∨ B) ∧ (A ∨ C)

DPLL

An algorithm for deciding satisfiability of propositional formulas.

It has:

A literal is a proposition or atomic statement or negation of the proposition

- From proposition A you get two literal
  - ex. A
  - ex. ¬A
- Pure literal
  - literal that occurs in only one clause

A variable is one of the simple statements (the letters) that make up the input statements

- The name of a statement, it doesn't include negation

A clause is a disjunction of literals

- Clauses are the building blocks of CNF
- Each parenthesized group is a clause most of the time
- Single-literal clause (unit clause)
  - ex. P
- Multi-literal clause
  - ex. (¬P ∨ Q)

The polarity of a literal indicates whether its positive or negative

- Positive polarity → not negated
  - ex. A, B, C
- Negative polarity → negated
  - ex. ¬A, ¬B

The easy cases of DPLL are:

- If a clause contains ¬A ∨ A it can be removed, because the case will be satisfied in any case
- If there is a pure literal then that literal can be set to true, because this choice doesn't affect any other clauses
- If there is a unit clause that literal can be set to true, because this is the only way the clause can be satisfied (ex. if the literal ¬A is set to true, it will become false)

Rules of DPLL:

- Unit propagation: the literal in a unit clause must be true, otherwise the clause can't be satisfied
- Pure literal elimination: set the literal so that the polarity is true (ex. False for ¬A and True for A)

- Branching (decision): If nothing forced remains pick a variable and try it out with either True or False, recursively run DPLL. If that branch returns unsatisfied backtrack and try the other one.
- If all clauses become satisfied → SAT and if an empty clause appears → UNSAT

Example 1:

(A ∨ B) ∧ (¬A ∨ C) ∧ (¬B ∨ C) ∧ (¬C ∨ D)

Clauses:

- C1: A ∨ B
- C2: ¬A ∨ C
- C3: ¬B ∨ C
- C4: ¬C ∨ D

Step 1: Check Unit Clauses

- None (no clause is the size of 1 literal)

Step 2: Check Pure Literals

- A appears only as A in C1 and as ¬A in C2 — both polarities → not pure.
- B appears B in C1 and ¬B in C3 → not pure.
- C appears only as C in C2 and C3 and only as ¬C in C4 → both polarities → not pure.
- D appears only as D in C4 and never as ¬D → D is a pure literal (positive).

Which means D = True, so you can drop C4

Step 3: Branch (Decision)

Try A = True

- C1: (A ∨ B) is satisfied (drop it).
- C2: (¬A ∨ C) → (False ∨ C) → becomes unit (C) → so C = True.
- C3: (¬B ∨ C) → with C = True becomes satisfied (drop it).

No conflict → SAT found

* B is unconstrained (not forced by unit clauses or needed to satisfy any remaining clause), so you can assign it True or False, whatever you want, it won't make a difference in the satisfiability.

Example 2:

(A ∨ B) ∧ (¬A ∨ C) ∧ (¬B ∨ C) ∧ (¬C ∨ D) ∧ (¬D)

Clauses:

- C1: A ∨ B
- C2: ¬A ∨ C
- C3: ¬B ∨ C
- C4: ¬C ∨ D
- C5: ¬D

Step 1: Check Unit Clauses

- Yes → C5 = ¬D

- Unit propagation: set D to False
- C4: (¬C ∨ D) becomes (¬C ∨ False) → **(¬C)** — now a unit clause.
- Others unchanged.

- Yes → C4 = ¬C
  - Unit propagation: set C to False
  - C2: (¬A ∨ C) → (¬A ∨ False) → (¬A) → unit → set A = False.
  - C3: (¬B ∨ C) → (¬B ∨ False) → (¬B) → unit → set B = False.
  - C1: (A ∨ B) → (False ∨ False) → empty clause (contradiction)

Empty clause → UNSAT

*The other steps weren't needed for this example

12th of November (WC)

A tautology means everything in the row is true

If an entailment of two rows are the same with the true and false then it means that the formula is entailed

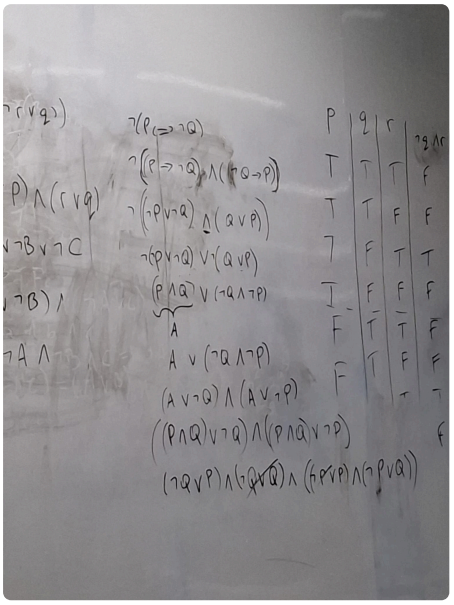
|= entailment sign

∧ and , is the same thing

When the ∧ are in the brackets, it's not CNF (the final state of the formula). The same when a ¬ is outside of the brackets.

DPLL:

1. Start from a ground CNF formula



2. Try to build an assignment that varies the formula
3. The assignment is build using a backtracking mechanism
4. If all clauses are satisfied, the formula is satisfiable with the current assignments
5. If all possible assignments have been tried and a contradiction was reached each time, the formula is unsatisfiable

17th of November (HC)

Lecture 7: Modeling Schnapsen with Propositional Logic Rule-based Reasoning & Knowledge Graphs

Propositional Logic

Aka Boolean Logic deals with propositions - statements can be either true or false, these proposition can be connected using logical connectives:

- ¬ (NOT)
- ∧ (AND)
- ∨ (OR)
- → (IMPLIES)
- ↔ (IFF)

It has a simple knowledge representation method, everything is presented as true/false statements and it uses deductive reasoning

PROs

- Declarative
- Compositional
- Allows partial/disjunctive/negated information, unlike most data structures
- Meaning is context-independent, unlike natural language

CONs

- Limited expressive power, unlike natural language

ex. “pits cause breeze in adjacent squares” can only be written using one sentence for each square

→ too complicated for Schnapsen

→ not meant to deal with partial observability

Inference Algorithms

Allow an AI system to derive logically valid conclusions from a knowledge base, they try to determine whether KB |= a (a is entailed by the KB)

1. Truth Tabel Enumeration
    - Check all possible assignments of truth values to propositions
        - First variable: TTFF, second variable: TFTF
        - First variable: TTTTFFFF, second variable: TTFFTTFF, third variable: TFTFTFTF
        - If a is true in every model where KB is true → KB |= a

    -Not usable for large problems
2. Inference by Resolution
    - Proving entailment via contradiction
    - Needs CNF conversion and uses a single rule
        - A ∨ B and ¬A ∨ C = B ∨ C
            - because A and ¬A are a contradiction and you can just delete those, this means KB entails B ∨ C

    -Can still be slow without optimization
3. Forward Chaining (Data-driven Interference)
    - Starts with known facts
    - Repeatedly apply rules whose premises (if parts) are already known to be true

- When a match is found, the conclusion (then part) of that rule is added as a new rule until the query (goal) is reached

-Not good when the goal is specific cause it has to explore everything

4. Backward Chaining (Goal-driven Interference)
   - Starts with the query (goal) that needs to be proven
   - Try to prove it using rules whose conclusion (then part) matches the goal
   - If a matching rule is found, the premises (if parts) of that rule become the new subgoals to prove
   - Recursively solve subgoals

   -Can loop without memorization

   +Doesn't explore irrelevant facts
5. DPLL Algorithm
   - Unit propagation
     - If a clause has one literal left, that literal must be true
   - Pure literal elimination
     - If a literal always appears with same polarity, assign it true
   - Backtracking search

   +Extremely efficient

Knowledge graphs

A structured model for representing real-world entities (like people, places or concepts) and the relationships between them, it's a large interconnected network of facts. They're mathematically modeled using graph theory with an easily readable and processable graph structure for machines. KG's capture the meaning and context of data, not just the keywords.

Triple Syntax → RDF

The information of the Knowledge Graphs is stored in triples:

1. Nodes (entities)
   - The nodes (or vertices) represent the entities - the real word objects or concepts
     - ex. "Albert Einstein", "Germany", "The theory of relativity"
2. Edges (relationships/predicates)
   - The edges (or links) represent the relationships (or predicates) that connect two entities. Edges are typically directed, showing the flow of the relationship
     - ex. "was born in", "is the author of", "invented"
3. Triples (Subject-Predicate-Object)
   - A fact is stored as a triple, linking a subject (node) to an object (node) via a predicate (edge)
     - ex. "Albert Einstein" → was born in → "Germany"

Ontology

A structured way of defining the types of entities, properties, and relationships that exist in a specific area of knowledge (a domain). It acts as a map of concepts that allows people, systems, and machines to communicate and share information unambiguously.

Knowledge Engineering

The art and science of transforming human knowledge (often vague, implicit, or unstructured) into a form that machines can process and reason with. Core steps in Knowledge Engineering are:

1. Knowledge Acquisition

    - The process of extracting knowledge from various sources (ex. human experts, documents, data mining)
2. Knowledge Representation
    - Choosing and formalizing the acquired knowledge into a structured format suitable for machine processing, common methods:
        - Rules: using IF-THEN statements
        - Frames/objects: Defining entities and their relationships
        - Logic: using propositional or first-order logic
        - Knowledge graphs: Using the triple syntax and ontologies to define entities and their relationships
3. Knowledge Formalization and Reasoning
    - Implementing the structured knowledge within a system and providing mechanisms for inference
        - Inference algorithms: using those to derive new conclusions from the known facts and rules
        - Consistency checking: ensuring the knowledge base does not contain contradictory rules or facts
4. Knowledge Maintenance
    - As the real world changes and new discoveries are made, the knowledge base must be updated, tested, and expanded. This is an ongoing process crucial for the system's longevity and accuracy

18th of November (HC)

Not needed for the midterm!!

19th of November (WC)

Closure

KB* = initial known facts (KB) + inferred facts

→ inferred facts are the rules you used and the steps you took to get to the answer. To add inferred rules you use CNF

In CNF only the new formula is enough but with knowledge bases we need to mention which rules we used.

If the question is does the KB entail C, and C is not in the KB on its own but with some other things in a clause, the answer is no it doesn't entail C.

Modus Ponens

If a conditional statement (If P then Q) is known to be true, and its antecedent (P) is also known to be true, then its consequent (Q) can be inferred to be true.

Triple

Structure composed of three elements, typically a subject, a predicate and an object.

→ the subject is the first bubble, the predicate is the arrow that points to the other bubble, which is the object.

Rule to Infer Triples

The type of the subclass of a class, is a type of the class as well

- a type b & b subclass c → a type c

Graph notation and Structure

- The graph is represented as a pair (V, E), where V is the set of vertices (or nodes) and E is the set of edges. In this case, V = {a, b} represents the two nodes in the graph
- The set of edges is E = {(a, b) (a, a)}. Each edge is represented as a pair, indicating a connection from the first element in the pair to the second.

Subgraph

Subset of a graph's vertices and edges

Unconnected Subgraphs

Two subgraphs are unconnected if there are no edges between any of the vertices in one subgraph and any of the vertices inn the other subgraph

Directed Graph

A directed graph has edges where the order matters; that is, an edge from node A to node B is different from an edge from node B to node A. In notation, these are different than (A, B) =/ (B, A). They're indicated with the curly brackets { }

Undirected Graph

An undirected graph ???

DFS → LIFO list

BFS → FIFO queue

22nd of November

IS Online Exam Prep

Uninformed search

- When you don't have any information about how far you are from the goal node.

A tree is a graph with no cycles (no edges between siblings)

Leaves are nodes without children.

The degree is the maximum amount of children in the tree, the depth or height are the layers after the root node

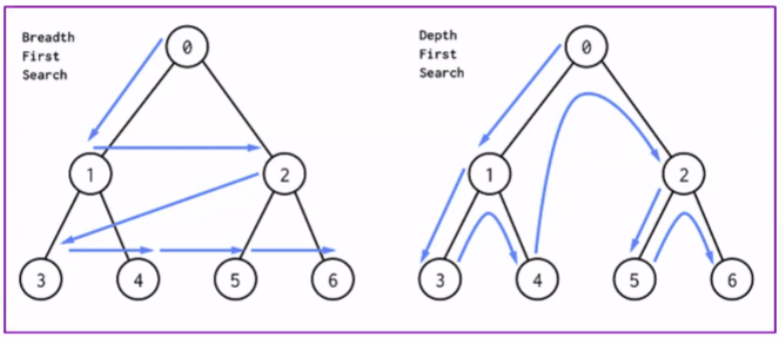
- height always checks the last node, depth can be of a specific node, the depth of a tree however is the same as the height.

Breadth First Search (BFS):

- FIFO queue
  - children are added at the end of the fringe
- Optimal
- Complete
  - It will always find a solution if there is one
- You go from left to right usually

Depth First Search (DFS)

- LIFO stack
  - Children are added to the beginning of the fringe
- Not optimal
  - Because it goes really deep
- Not complete
  - It can get stuck in loops if it is really deep



Depth-First Iterative Deepening Search

- Combination DFS & BFS
- Optimal
- Complete

Do not repeat the nodes if you explore them more times, don't write them down again. You go per depth and check all the nodes in that depth.

- The difference between DFIS and BFS is the way you do it, you go all the way back and repeat all the nodes in DFIS but you don't write the repeated nodes down, which means that the way you write it down is the same as BFS

Depth Limited

- When the limit is set at depth = 1, and the goal is in a lower depth, you won't find it

---

Informed Search

Heuristic value: f(n) = g(n) + h(n)

Hill climbing

You stop when there are no better nodes after the one where you are

- You can't backtrack

Greedy best-first search

You go back to every node that is better or equal than the one where you are

- You can backtrack

*A & A**

The difference is that in A* the heuristics is admissible

Algorithm	Keeps nodes in memory	Optimal?	Complete?
Hill climbing	No	No	No
Greedy BFS	Yes	No	No
A & A*	Yes	Yes	Yes

18th of November (HC)

Lecture 8: Vague and uncertain knowledge

Vagueness vs uncertainty

Uncertainty is a lack of information, where the provided information leaves doubt about its accuracy or completeness, whereas vagueness is a lack of definition (you have the information, but the definitions are blurry), there is room interpretation.

- If you are uncertain, you don't know the state of the world (but there is one). If you are dealing with vagueness, you know the state of the world perfectly, but your words don't fit it precisely.

Fuzzy sets

How do computers, who think in zeros and ones handle 'fuzzy' language:

1. The old way: Crisp Sets
    - Everything is a hard yes or no
    - If we define 190cm as tall, then 189cm wouldn't be considered as tall which doesn't make sense
2. The new way: Fuzzy Sets
    - It allows for degrees of truth, instead of just 0 *or* 1, you can choose *between* 0 and 1
    - It uses membership functions:
        - a mathematical function that calculates *how much* an object belongs to a set, it assigns a 'grade' or 'degree' of membership to every object.
        - Different shapes of membership:
            - Increasing notions: A line that goes up and stays up (like 'Tall' or 'Fast', because the higher you get the taller or faster you become)
            - Approximating notions: A triangle or bell shape. This is used for concepts like 'medium-sized', 'about-average' or 'near ...'
                - It starts at 0, goes up to a peak (1) at the perfect average, and then goes back down to 0 as you move away from the average
            - Piecewise linear: Often, complex curves are simplified into straight lines connected together (like a trapezoid) to make calculations easier for computers
3. Modifiers (Hedges)
    - You can use math to change these curves, if you want to define 'very tall', you shift the graph to the right
4. Operations: Doing math with fuzzy sets
    - NOT (Complement): subtract the value from 1 (if you're 0.8 Tall, you are 1 - 0.8 NOT Tall)
    - AND (Intersection): take the minimum (if you're 0.8 Tall AND 0.4 Heavy, the score for Tall AND Heavy is 0.4 (the lower number))
    - OR (Union): take the maximum (the score for Tall OR Heavy would be 0.8 (the higher number))

Fuzzy logic and relations

1. Fuzzy logics
   - In a standard list we might say John likes Movie 1 (yes/no), in a fuzzy list we say John likes Movie 1 to a degree of 0.7
2. Composition (The chain rule)
   - *If we know who likes which Movie, and we know which Movie features which Actor, can we figure out who likes which Actor?*
   - To find this out we use the Max-Min rule
     - Min: look at the chain, the connection is only as strong as the weakest link
     - Max: is there another movie John watched with the same actor, we calculate that chain too, then we pick the *strongest* (maximum) result among all the different movie chains

---

Uncertainty

Now we're talking about actually missing information, we're uncertain

Uncertainty arises from data that is:

1. Unreliable
   - data can be incorrect
2. Incomplete
   - existing data is correct but you're still missing something
3. Imprecise
   - they may be correct but inadequate for the level of detail required

---

Views on probability

The Frequentists

- They believe probability is a hard fact based on history
- ex. if you do an experiment 100 times and it happens 20 times, the probability is 20%

The Bayesians

- They believe probability is about your personal uncertainty or belief
- ex. "I'm 20% sure Bob is having an affair", you can't repeat Bob's life 100 times to test this theory, it is a subjective belief based on evidence.

*Even though they think differently, both of them use the same math formulas to solve problems

25th of November (HC)

Calculating with probability (Bayesian networks)

Basics of probability

Sample Space

- A set of all possible outcomes
  - ex. if you roll a dice the sample space is `{1, 2, 3, 4, 5, 6}`
- Every single item in the list is called an atomic event (or 'possible world')

Events

- Specific subset of the possibilities that we care about
  - Atomic event: ex. rolling a 6
  - Complex event: ex. rolling an even number `{2, 4, 6}`

Probability Space

- Here you assign a number between 0 and 1 to those events
  - 0 means it's impossible
  - 1 means it's certain
- If you add the probability of every possible atomic event, the sum must be exactly 1

Random variables and distributions

Random Variables

- Like a question we ask about the world, where the answer isn't fixed yet
  - Boolean: True/False
  - Discrete: a specific category (ex. `Weather = sunny` or `Weather = rainy`)
  - Continuous: a number (ex. `Temperature = 26.4`)

Probability Distributions

- A table that lists every possible answer for a variable and the probability of it happening
  - ex. in a card game, the distribution for the next card is 25% spade, 25% heart, 25% club and 25% diamond

The Joint Probability Distribution

- A big table that shows the probability of two or more things happening at the same time
  - ex. what is the chance that `Value > 4` and `Card = diamond`

Marginal Probability

- When you have a big table of complex data but you only look at one variable, ignoring the other
  - ex. a table with students passing and if they attended the lectures, but you only look at the amount that passed without keeping the fact if the attended the lecture in mind

---

Conditional probability

- What is the chance of A happening, given that we already know B happened?
- Notation: P(A | B)
- Formula: P(A | B) = P(A, B) / P(B)
  - The chance of both happening together divided by the chance of the condition happening

Inference by Numeration

- Calculating the answer by looking at the table and adding up the numbers
  - ex. if you want to know P(Cavity | Toothache), you look at the table, find all the rows where "Toothache" is true, and calculate what percentage of those rows also have "Cavity"

Bayes' Theorem

- It solves the Inversion problem

  → Usually we have data on the effect given the cause

- *We know:* If you have the Flu (Cause), there is an 80% chance you have a fever (Effect)
- *We WANT to know:* If a patient has a fever, what is the chance they have the Flu?

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

- P(A|B): The answer we want (Chance of Disease given Symptom)
- P(B|A): The data we have (Chance of Symptom given Disease)
- P(A): The Prior. How common is the disease in general?
- P(B): The Evidence. How common is the symptom in general?

Bayesian Networks

- We can't make a giant table for all the variables in the world, because it will be too big
- So, instead of 1 big table, we draw a diagram (a graph) that actually shows what affects what
  - Nodes: the circles represent random variables
  - Arrows: the lines show dependencies, the way it points indicates what affects what

Conditional Independence

- We assume that if two nodes are not connected, they don't directly talk to each other
  - Some variable affect each other through other variable, not directly between each other, this means they're conditionally independent

Calculation Formula

- To find the probability of something, you multiply the probability of each event given its parents

26th of November (WC)

Vagueness is when something lacks clarity or precision, that leaves room for interpretation (ex. Dutch men are very tall)

Uncertainty is when the provided information leaves doubt about its accuracy or completeness, it implies that there is something not fully known within the context of the statement (ex. half of the Dutch men are married)

Fuzzy set theory is when elements can belong to a set to a certain degree

Membership function gives a number between 0 and 1, where 0 means no membership and 1 means full membership

The peak of a graph is considered the ideal situation

When the P is bold (**P**) it's a a vector probability, this means that we look at both the case where it's true and where it's false, when the P is normal you just look at when it's true (the marginal probability)

1st of December (HC)

Machine Learning 1

The basics of classification

- Imagine you have a new unlabeled piece of data and you need to put a label on it, this is classification

We use math, the Bayes' Theorem, to calculate the odds

→ A machine learning model looks at 'features' to predict a 'class'

Introduction to Machine Learning

Adaptive Intelligent Agents

- They respond to their environment or other agents and learn from these interactions over time (learns from past experience to improve future behavior)

ML vs. AI vs. Deep Learning

AI is the big umbrella that includes anything that makes a machine smart, ML is a subset of AI which requires the system to learn and adapt from experience rather than just following rules. Deep Learning is a subset of ML and uses neural networks inspired by the human brain

Types of Learning

- Supervised learning: We act as a teacher, we give the computer data and the correct answers (labels)
- Unsupervised learning: We give the computer data without the answers, it must find patterns on its own, like grouping similar items together
- Semi-supervised learning: A mix of both, we define some items but leave others blank
- Reinforcement learning: Learning by trial and error, the computer gets a 'reward' for good behavior and 'punishment' for bad behavior#

Common ML Tasks

- Classification: Sorting things into categories or labels (SL)
  - It predicts a discrete class rather than a number or a ranking
- Regression: Predicting a specific number (SL)
  - It outputs a specific number rather than a category
- Ranking: Ordering items by relevance (SL)
  - It focuses on the relative order of items rather than their individual values or classes
- Collaborative filtering: Recommendation systems using patterns in user interactions (SL)
  - It relies on user-to-user or item-to-item similarities rather than analyzing the content of the items themselves
- Clustering: Grouping similar items without any prior labels (UL)
  - It finds patterns on its own without knowing what the 'correct' groups are beforehand

---

Specific Algorithms

Naive Bayes Classifier

- It calculates the probability that a data point belongs to a category based on its features
- It assumes that features are independent of each other

- It multiplies all the probabilities with each other and then compares the outcome with the outcomes for different classes
- Because of the multiplication you need to make sure that a probability is never 0, otherwise your outcome will always be 0, because of this you always a number, like 1, this is called smoothing

k-Nearest Neighbor (k-NN)

- This is a visual method, plotting all your data points on a graph
- Things that are similar usually sit close together on the graph

1. Find the *k* closest dots to the new mystery dot (k is a number)
2. Check the labels of those neighbors
3. If the majority of the labels is one thing, then that's also the label for the mystery dot

- If you only look at one neighbor, it's not reliable, but by looking at a group you can be more certain

2nd of December (HC)

Machine Learning 2

Machine Learning Methodology

You need to organize your data, this is typically done into three chunks:

- Training set: The big chunk of data used to actually teach the model
- Validation set: A smaller chunk used to finetune the settings (hyperparameters) of the model
- Test set: Data the model has never seen before, we use this at the very end to see if the model actually works

Features

- List of numbers that computers use, that are normal words for us

---

Understanding Text

We can turn a sentence into a vector (a list of numbers). ex. if a word appears in the list you can assign it a 1 and if it doesn't a 0. Once the sentences are turned into vectors we can calculate how similar they are using math formulas

- Hamming distance: Counts how many bits (0s and 1s) are different between two lists
- Euclidean distance: Measures the straight line distance between two points on a graph
- Cosine similarity: Measures the angle between the two vectors

To make it easier for the computer, we 'clean' the text features

- We make everything lowercase
- We chop off the ends of words, go back to the root of the verb or make words singular
- We delete common, useless words like 'the', 'and' or 'it' because they do not contribute to the final answer

---

Grading the model

- Cross-Validation: if we don't have a lot of data, we can split it into test groups multiple times to double check our error rate
- Confusion Matrix: this is a table that shows us exactly how a model failed. It tracks:

    - True Positives (TP)
    - False Positives (FP
    - True Negatives (TN)
    - False Negatives (FN)
- Accuracy: the percentage of times the model was correct overall

Evaluating Other Tasks

- Regression: if we are predicting a number instead of a category, we use Mean Squadded Error (MSE), this measures the average distance between our predicted and real number

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Ranking: If we are recommending movies we use precision@k measure, this checks if the 'good' movies appeared in the top few recommendations

$$\text{Precision@}k = \frac{true\ positives\ @k}{(true\ positives\ @k) + (false\ positives\ @k)}$$

3rd of December (WC)

Classification errors are when a model incorrectly predicts the category or class of a data point

Precision is a metric that evaluates the proportion of true positive predictions among all positive predictions made by a model

- It measures the quality
- *Of all the instances the model labeled as positive, how many are actually positive? (How many selected items are relevant?)*
- TP / (TP+FP)

Recall is a metric that evaluates the proportion of actual positives correctly identified by the model (which are false negatives)

- It measures the quantity
- *Of all the actual positive instances, how many did the model successfully identify? (How many relevant items are selected?)*
- TP / (TP+FN)

True Positives (TP) - true and predicted

False Positives (FP) - false but predicted

False Negatives (FN) - true but not predicted

True Negatives (TN) - false and not predicted

*If you need to decide which one is better, you need to take the scenario into account

---

Regression model is a model that predict numerical values based on input date

Regression errors are discrepancies between actual values and predicted values

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

This just means that you subtract the 2 values in a set and do that for every set that you got and that add those together.

If they ask for a better function, you want the outcome to be lower than the initial outcome, because that means that the error is smaller.

Ranking errors are discrepancies between predicted order of items and the true, expected order based on relevance or importance

Ranking model is a model that sorts items or data points in a specific order based on relevance relative to a given query or set of criteria

Ranking precision is a ratio of relevant items found up to that rank over number of items up to that rank

- All the levels to that point - the number of errors until that point / all the levels to that point

---

Bayesian learning for text classification

Naive Bayes classifier is a simple probabilistic classifier that applies Bayes' theorem with the assumption of independence between every pair of feature

- You calculate the possibility of the class you want to classify the variable in by dividing the variables that are already classified in that class by all the possible variables that could be classified in that class. Then you calculate the probability for all the factors that decide the classification by the Bayes' Theorem formula. And then you multiply everything with each other
  - Normally you have more than one class, so you do this for every class and the one with the highest outcome, highest probability, is the answer to the question

k-NN classifier is a non-parametric method used for classification and regression that predicts the label of a data point by looking at the 'k' closest labeled data points and choosing the most common label arguing them

*k3 means you need to choose the 3 smallest numbers, it would've been the 2 smallest number if it was k2 etc.

*If 2 numbers are the same amount, then you do it 2 times one with each of the numbers

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

okay i'm lost

Hamming Distance is the calculated amount in k-NN divided by the total amount

Cosine Similarity is calculated through:

$$\frac{\textit{how many times both have 1}}{\sqrt{\textit{number of 1 in our knowledge base}}} * \sqrt{\textit{number of 1 in our example}}$$

8th of December (HC)

Machine Learning 3

Model Class Selection for Classification

Steps required to build a machine learning system:

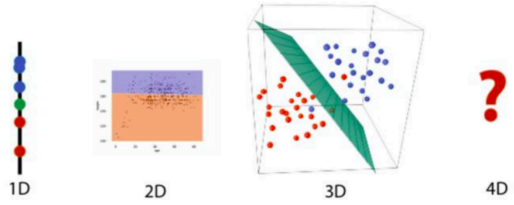
- Choose features: decide what data characteristics to use (ex. word count or sentence length for text)
- Choose model class: deciding what 'shape' or type of model fits the data
- Choose search method: how to optimize that model (finding the best fit)

The Linear Classifier

Imagine a dataset where you want to classify something into one of two groups, the simplest way to separate those two groups is to draw a straight line through the data graph. Everything above the line belongs to one group and everything beneath to the other. A single line only works where there are only 2 features (like age and height), when there are more features it's called a hyperplane.

Hyperplanes

- 1D - If you only have one feature, the separator is just a single point
- 2D - If you have two features, the separator is a straight line (the linear classifier)
- 3D - If you have three features, you can imagine a flat sheet of paper (a plane) slicing through a 3D cloud of data points to separate them
- 4D - When you have 4 or more features, we can no longer visualize it, the separator is called a Hyperplane



In the 2D model each of the axes represent a feature, and in the 3D model each side (length, width, height) represents a feature

Formula for the 4 dimensional hyperplane: $ax_1 + bx_2 + cx_3 + dx_4 + e > 0$

- $x_1, x_2 ...$ are the features
- *a, b, c, d* are the weights the model learns, they determine the angle/orientation of the hyperplane
- *e* is the intercept, it acts as an offset, pushing the hyperplane up or down so it doesn't have to pass through the origin
- The computer calculates this sum, if the result is positive, the data point belongs to one group and if it's negative it belongs to the other

The N-1 Rule: *"The hyperplane is one dimension less than the dimensions of the feature space"*

- In a 3D-space, the separator is a 2D plane
- In a 2D-space, the separator is a 1D plane, a straight line

Maximum Margin Hyperplane

Sometimes there are more than 1 plane separating the dots completely without 1 dot on the wrong side. We choose the line for which the distances to the nearest example of either class is the greatest, the line with the maximum margin to all the data points.

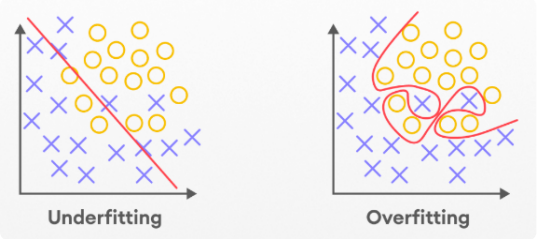
You take the line you want to check the margin distance from and you draw two parallel lines on either side touching the closest dot. The goal is to make the distance between your line and the support vector lines as great as possible. You want the distance to be big because then you're more likely to be corrects with new, unseen data.

Support Vector Machines (SVM) is a very famous and successful machine learning algorithm.

Not Linearly-Separable Data

When the dataset can't be divided by a single line, because there are lots of strays, we're tempted to implement circles or something else but this makes it more difficult to search for a good model. That is when you want to change the features so that the line does separate the dots correctly. When you don't have the resources to find new features you can combine the features that you have and put all possible outcomes in a design matrix. When you implement those results you'll still have a line only this time it will be more curved. We believe that are linearly separable if you project them into a space of high-enough dimension.

Under/Overfitting



With overfitting the model has the perfect performance on the training data, but when we draw new data from the same source the chances that the new dots are misplaced is very big. But with underfitting the classifier is too general and unspecific for the data.

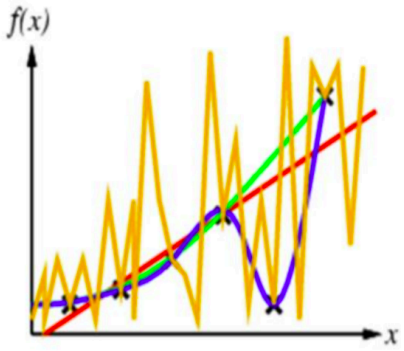
Model Class Selection for Regression

Regression is different from classification, instead of predicting a class (ex. Healthy vs. Sick), you predict a specific number (ex. a grade or height).

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

This just means that you subtract the 2 values in a set and square the outcome, do that for every set that you got and add those together.

When you need to find a better outcome, you change the m and b in the formula y = mx + b and use the x input to get the predicted answer from the model. Then you subtract the actual y from the predicted y and square this. Do this for every data point and add them, when the outcome is lower than the previous outcome the system is better.



The purple and the yellow line are consistent

You need to construct the hypothesis *h* so that it agrees with what is observed in the dataset *f*. *h* (the lines) is consistent if it agrees with *f* (the dots) on all examples. In the image above two lines are consistent but you choose the simplest one, the Occam Razor Principle (parsimony).

> *"One should not increase, beyond what is necessary, the number of entities required to explain anything"*

Finding the Optimal Model

1. Use the MSE formula
2. Calculate the derivative of that formula
3. Make the derivative equal to 0
4. Solve this sum and that's your answer

Gradient descent and stochastic descent can both be used to find the optimal. Gradient descent moves one step at a time but can get stuck in a local minimum which makes it harder for it to find the global minimum. Stochastic descent only uses 1 spot.

9th of December (HC)

Machine Learning 4

Clustering

With clustering, or specifically k-means clustering you want to group similar things together without knowing what they are named.

k is the number of clusters created, which are estimated by the user.

1. Choose *k* amount of points as the first seeds
2. Assign each data point to the cluster of the nearest random seed
3. The seed changes to the average point in each created cluster
4. Repeat step 2
5. Repeat step 3 and 2 until the average point of the cluster hardly or doesn't move

Clustering Evaluation

- External criterion: actual labels are known
  - evaluation is similar to classification
- Internal criterion: actual labels are unknown
  - You don't know if the model did a good job
  - Inter- & Intra-cluster similarity is used

- Goal is a high Intra-cluster (in the same cluster) similarity and a low Inter-cluster similarity (between two clusters)

$$a_i = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d(i, j)$$

You divide 1 by the total number of data points in the cluster i belongs to minus 1. Then you multiply this by the sum of the distances between point i and every other point j (points in the same cluster).

Artificial Neural Networks

A perceptron is computer version of a human neuron, it's simply several weights *w* and a bias *b* term as input features leading to some output *y*.

$$y = w_1 x_1 + w_2 x_2 + b$$

The problem with building deep networks with perceptrons is that these function will always be linear, even with many layers it can be simplifiable.

Non-linearities: Activation Functions

Sigmoid σ is an activation function with a characteristic S curve, it changes each value to be between 0 and 1.

In a feedforward network, weights and biases only flow forward in one direction, the input can be fed to hidden layers before going to the output. The more hidden layers, the deeper the network (multilayer perceptron).
Softmax is an activation function for multiclass classification, which gives a probability of an instance belonging to each class, this means that there is more than one output. The softmax function assigns a probability to each class, classifying an instance as the class with the highest probability, the result is a valid probability distribution over classes.

Backpropagation is making a backward pass to adjust a network's parameters.

Deep Learning & Transformers

Transformer Architecture

This architecture is designed for sequence data, like text or speech. This model is often trained on next-word prediction, making it a language model. It reads the whole sentence at once, not forgetting the first words by the time they reach the end like computers usually do. It pays attentions to how every word relates to every other word simultaneously.